PROBLEM OF THE DAY

POTD — Arrays, Objects & Higher-Order Functions

12 interview-style questions — mixed MCQ & MSQ — built around a single API-style products array.

Mid → High difficulty · Array · Object · forEach · map · filter · reduce

HOW TO ATTEMPT

Format for today's set

MCQ

Single correct answer.

Exactly one option (A–D) is right — pick that one.

MSQ

Multiple correct answers.

One or more options can be right — select every option that applies.

Q → A

Question, then answer.

Each question sits on its own slide. The very next slide reveals the answer — attempt it before you look.

All twelve questions are self-contained in their code, except where they explicitly reference the products array on the next slide.

REFERENCE DATA

The products array

Several questions in this set operate on this exact array — keep it in mind as you go.

products.js

```
// products API response (reference for this POTD)
const products = [
  { id: 101, name: "Wireless Mouse", category: "Electronics", price: 799,  quantity: 2,  inStock: true  },
  { id: 102, name: "Yoga Mat",       category: "Fitness",     price: 499,  quantity: 0,  inStock: false },
  { id: 103, name: "BT Speaker",     category: "Electronics", price: 1999, quantity: 5,  inStock: true  },
  { id: 104, name: "Running Shoes",  category: "Fitness",     price: 2999, quantity: 1,  inStock: true  },
  { id: 105, name: "Coffee Mug",     category: "Home",        price: 299,  quantity: 10, inStock: true  },
];
```

Q1

MCQ · single answer

What does result hold after this runs?

snippet.js

```
const nums = [1, 2, 3];
const result = nums.map((n) => {
  n * 2;
});
console.log(result);
```

A [2, 4, 6]

B [undefined, undefined, undefined]

C [1, 2, 3]

D TypeError: n * 2 is not a function

ANSWER 1

MCQ · single answer

Correct: B

snippet.js

```
const nums = [1, 2, 3];
const result = nums.map((n) => {
  n * 2;
});
console.log(result);
```

A [2, 4, 6]

B [undefined, undefined, undefined]

C [1, 2, 3]

D TypeError: n * 2 is not a function

WHY

The arrow function has a block body ({ ... }) but no explicit return, so it returns undefined on every call. map() always produces a new array the same length as the input, filled here with three undefined values — it doesn't care what happens inside the callback body.

Q2

MCQ · single answer

What gets logged to the console?

snippet.js

```
const ids = [10, 20, 30, 40];
ids.forEach((id) => {
  if (id === 20) return;
  console.log(id);
});
```

A Prints 10, 20, 30, 40 — return is ignored

B Prints 10, then stops completely (like break)

C Prints 10, 30, 40 — return only skips that iteration

D Throws SyntaxError: illegal return statement

ANSWER 2 MCQ · single answer Correct: C

snippet.js

```
const ids = [10, 20, 30, 40];
ids.forEach((id) => {
  if (id === 20) return;
  console.log(id);
});
```

A Prints 10, 20, 30, 40 — return is ignored

B Prints 10, then stops completely (like break)

C Prints 10, 30, 40 — return only skips that iteration

D Throws SyntaxError: illegal return statement

WHY

return inside a forEach callback exits only that single call — like continue in a for loop — not the whole iteration. forEach has no built-in way to break early; for that you'd need a for loop, some(), or every().

Q3

MSQ · select all that apply

Which of these array methods do NOT mutate the original array?

- `forEach`
- `map`
- `filter`
- `reduce`
- `sort`
- `splice`
- `push`

ANSWER 3

MSQ · select all that apply

Correct: A, B, C, D

- ✓ `forEach`
- ✓ `map`
- ✓ `filter`
- ✓ `reduce`
- `sort`
- `splice`
- `push`

WHY

forEach, map, filter, and reduce always leave the original array untouched — map/filter/reduce return new values and forEach returns undefined. sort() and splice() rearrange or remove elements in place, and push() adds to the original array directly.

Q4

MCQ · single answer

What happens when this code runs?

snippet.js

```
const empty = [];
const total = empty.reduce((acc, x) => acc + x);
```

A total is 0

B total is undefined

C Throws TypeError: Reduce of empty array with no initial value

D total is NaN

ANSWER 4 MCQ · single answer **Correct: C**

```
snippet.js
const empty = [];
const total = empty.reduce((acc, x) => acc + x);
```

A total is 0

B total is undefined

C **Throws TypeError: Reduce of empty array with no initial value**

D total is NaN

WHY

Without an initialValue, reduce() tries to use the array's first element as the starting accumulator. On an empty array there is no first element, so it throws instead of silently returning 0. Always pass an initialValue when the array could be empty.

Q5

MCQ · single answer

Using the products array, what does cheapNames log?

snippet.js

```
const cheapNames = products
  .filter((p) => p.price < 1000)
  .map((p) => p.name);

console.log(cheapNames);
```

A ["Wireless Mouse", "Yoga Mat", "Coffee Mug"]

B ["Wireless Mouse", "Yoga Mat", "BT Speaker", "Coffee Mug"]

C ["BT Speaker", "Running Shoes"]

D [] — map already discarded the price field

ANSWER 5 MCQ · single answer Correct: A

snippet.js

```
const cheapNames = products
  .filter((p) => p.price < 1000)
  .map((p) => p.name);

console.log(cheapNames);
```

A **["Wireless Mouse", "Yoga Mat", "Coffee Mug"]**

B ["Wireless Mouse", "Yoga Mat", "BT Speaker", "Coffee Mug"]

C ["BT Speaker", "Running Shoes"]

D [] — map already discarded the price field

WHY

filter() runs first, keeping only Wireless Mouse (799), Yoga Mat (499), and Coffee Mug (299) — all under 1000. map() then runs on that already-narrowed array, pulling out just the name field from those three survivors.

Q6

MSQ · select all that apply

Which snippet(s) correctly compute total inventory value (price × quantity, summed)?

A

```
products.reduce((sum, p) =>
  sum + p.price * p.quantity, 0);
```

B

```
products.reduce((sum, p) =>
  sum + p.price * p.quantity);
```

C

```
let sum = 0;
products.forEach((p) => {
  sum += p.price * p.quantity;
});
```

D

```
const sum = products.map(
  (p) => p.price * p.quantity);
```

ANSWER 6

MSQ · select all that apply

Correct: A, C

A

```
products.reduce((sum, p) =>
  sum + p.price * p.quantity, 0);
```

B

```
products.reduce((sum, p) =>
  sum + p.price * p.quantity);
```

C

```
let sum = 0;
products.forEach((p) => {
  sum += p.price * p.quantity;
});
```

D

```
const sum = products.map(
  (p) => p.price * p.quantity);
```

WHY

A works: initialValue 0 gives a clean numeric start. C works: sum is declared as a number outside, then accumulated via forEach's side effect. B skips initialValue, so the first product OBJECT becomes the starting accumulator — object + number breaks the math. D returns an array of per-item totals, not a single summed total.

Q7

MCQ · single answer

What does list contain?

snippet.js

```
const stock = { mouse: 2, mat: 0, speaker: 5 };
const list = Object.entries(stock)
  .map(([name, qty]) => `${name}:${qty}`);
console.log(list);
```

A ["mouse:2", "mat:0", "speaker:5"]

B [["mouse",2], ["mat",0], ["speaker",5]]

C { mouse: "2", mat: "0", speaker: "5" }

D TypeError: Object.entries is not a function

ANSWER 7 MCQ · single answer Correct: A

snippet.js

```
const stock = { mouse: 2, mat: 0, speaker: 5 };
const list = Object.entries(stock)
  .map(([name, qty]) => `${name}:${qty}`);
console.log(list);
```

A **["mouse:2", "mat:0", "speaker:5"]**

B [["mouse",2], ["mat",0], ["speaker",5]]

C { mouse: "2", mat: "0", speaker: "5" }

D TypeError: Object.entries is not a function

WHY

Object.entries() turns an object into an array of [key, value] pairs. map() destructures each pair as [name, qty] and returns a template string — the final result is a flat array of formatted strings, not the pairs themselves and not an object.

object → Object.entries() → array of [key, value] pairs → map() → array of strings

Q8

MSQ · select all that apply

Which snippet(s) correctly group product names by category into one object?

A

```
products.reduce((groups, p) => {
  groups[p.category] =
    groups[p.category] || [];
  groups[p.category].push(p.name);
  return groups;
}, {});
```

B

```
products.reduce((groups, p) => {
  groups[p.category].push(p.name);
  return groups;
}, {});
```

C

```
products.map((p) => {
  const groups = {};
  groups[p.category] = [p.name];
  return groups;
});
```

D

```
let groups = {};
products.forEach((p) => {
  groups[p.category] =
    groups[p.category] || [];
  groups[p.category].push(p.name);
});
```

ANSWER 8

MSQ · select all that apply

Correct: A, D

A

```
products.reduce((groups, p) => {
  groups[p.category] =
    groups[p.category] || [];
  groups[p.category].push(p.name);
  return groups;
}, {});
```

B

```
products.reduce((groups, p) => {
  groups[p.category].push(p.name);
  return groups;
}, {});
```

C

```
products.map((p) => {
  const groups = {};
  groups[p.category] = [p.name];
  return groups;
});
```

D

```
let groups = {};
products.forEach((p) => {
  groups[p.category] =
    groups[p.category] || [];
  groups[p.category].push(p.name);
});
```

WHY

A and D both initialize each category's bucket before pushing into it, and both accumulate into ONE shared object. B throws — groups[p.category] is undefined the first time a category appears, so .push() fails. C builds a fresh {} on every call and map() returns an array of separate tiny objects, never one combined result.

Q9

MCQ · single answer

What does this function log?

snippet.js

```
function doubleAll(arr) {
  const result = arr.forEach((n) => n * 2);
  return result;
}
console.log(doubleAll([1, 2, 3]));
```

A [2, 4, 6]

B undefined

C [1, 2, 3]

D Throws ReferenceError

ANSWER 9

MCQ · single answer

Correct: B

snippet.js

```
function doubleAll(arr) {
  const result = arr.forEach((n) => n * 2);
  return result;
}
console.log(doubleAll([1, 2, 3]));
```

A [2, 4, 6]

B undefined

C [1, 2, 3]

D Throws ReferenceError

WHY

forEach() always returns undefined, no matter what its callback computes. Assigning that return value and handing it back is a classic bug — this function needed map() instead, which would actually collect [2, 4, 6].

Q10

MCQ · single answer

What gets logged?

snippet.js

```
const data = [1, 2, 3];
console.log(typeof data, Array.isArray(data));
```

A "array", true

B "object", true

C "object", false

D "array", false

ANSWER 10 MCQ · single answer Correct: B

snippet.js

```
const data = [1, 2, 3];
console.log(typeof data, Array.isArray(data));
```

A "array", true

B "object", true

C "object", false

D "array", false

WHY

typeof an array is always "object" in JavaScript — arrays are a specialized kind of object, and typeof can't tell them apart from a plain {}. Array.isArray() is the reliable way to check, and correctly returns true here.

Q11

MSQ · select all that apply

Given on `values.filter(Boolean)` `[0, "hello", "", null, 42, false, NaN, {}]` — which values get REMOVED?

- `0`
- `"hello"`
- `""` (empty string)
- `null`
- `42`
- `false`
- `NaN`
- `{}` (empty object)

ANSWER 11

MSQ · select all that apply

Removed: 0, "", null, false, NaN

0 ✓	"hello"	"" (empty string) ✓	null ✓
42	false ✓	NaN ✓	{} (empty object)

WHY

filter(Boolean) keeps only truthy values. 0, "", null, false, and NaN are all falsy, so they're removed. "hello" and 42 are truthy and survive — and the classic trap: {} (an empty object) is ALSO truthy in JavaScript, so it survives too.

Q12

MCQ · single answer

What does this log — and why?

snippet.js

```
const original = [{ id: 1, specs: { color: "red" } }];
const copies = original.map((item) => ({ ...item }));
copies[0].specs.color = "blue";
console.log(original[0].specs.color);
```

A "red" — map() produced a fully independent deep copy

B "blue" — the spread only shallow-copies; nested objects stay shared

C undefined — spread strips nested properties

D TypeError — you can't spread objects containing objects

ANSWER 12 MCQ · single answer Correct: B

snippet.js

```
const original = [{ id: 1, specs: { color: "red" } }];
const copies = original.map((item) => ({ ...item }));
copies[0].specs.color = "blue";
console.log(original[0].specs.color);
```

A "red" — map() produced a fully independent deep copy

B "blue" — the spread only shallow-copies; nested objects stay shared

C undefined — spread strips nested properties

D TypeError — you can't spread objects containing objects

WHY

{ ...item } only copies item's own top-level properties. specs is copied BY REFERENCE, so copies[0].specs and original[0].specs still point to the same nested object. Mutating one mutates both — map() gives you new top-level objects, not a deep clone.

THAT'S A WRAP

12/12 done.
See you tomorrow.

- Re-derive every wrong answer yourself — rewrite the snippet until its output matches your prediction.
- Notice the recurring traps: missing return in map, forEach's undefined return, missing reduce initialValue, and shallow copies.
- Tomorrow: chaining these methods together and reasoning about time complexity.